

SQL Fundamentals - Understanding SQL Syntax

Lecture 21

Innocent Nyalala

IIT Madras Zanzibar

Tuesday, May 20, 2025

Introduction to SQL Syntax

Today, we explore the core of SQL: its **syntax**, organized into five categories to understand how we interact with databases.

- SQL follows a specific set of rules called **syntax**.
- ◊ Statements start with keywords (e.g., SELECT, CREATE) and end with a semicolon (;).
- SQL is **case-insensitive** for keywords, but table names in MySQL are case-sensitive.
- ★ Categories: **DDL** (structure), **DML** (data manipulation), **DQL** (querying), **DCL** (permissions), **TCL** (transactions).

Focus

We will cover key SQL commands using a Football table, following a logical flow from database creation to querying and transaction management.

SQL Command Categories

SQL commands are grouped into five categories, reflecting different database operations:

- **DDL (Data Definition Language):** Defines and modifies database structures (e.g., CREATE, ALTER, DROP).
- ◊ **DML (Data Manipulation Language):** Manipulates data within tables (e.g., INSERT, UPDATE, DELETE).
- **DQL (Data Query Language):** Retrieves data (e.g., SELECT).
- ★ **DCL (Data Control Language):** Manages access and permissions (not covered today).
- **TCL (Transaction Control Language):** Manages transactions (e.g., COMMIT, ROLLBACK).

Logical Flow

We start with creating a database (DDL), populate it (DML), query it (DQL), and manage transactions (TCL).

DDL: CREATE DATABASE Statement

Purpose: Creates a new database to store tables and data.

Syntax

```
CREATE DATABASE database_name;
```

Example

```
-- Create a new database  
CREATE DATABASE FootballDB;
```

- Creates a new database named FootballDB to hold our Football table.

DDL: USE Statement

Purpose: Selects a database to work with for subsequent operations.

Syntax

```
USE database_name;
```

Example

```
-- Select the FootballDB database  
USE FootballDB;
```

- Sets FootballDB as the active database for creating tables and running queries.

The Football table stores information about football players, used for our examples.

- **PlayerID:** Unique identifier (auto-incrementing integer).
- ◇ **Name, Position, Nationality:** Variable-length text fields.
- **Age:** Whole number for player age.
- ★ **Price:** Decimal value for transfer fees (up to 10 digits, 2 decimal places).
- **ContractExpiryDate:** Date for contract end, can be null.
- ◇ **Constraints:** Ensures age is between 16 and 45, price is non-negative.

DDL: CREATE TABLE Statement

Purpose: Creates a new table with specified columns and constraints.

Syntax

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    ...  
    columnN datatype,  
    PRIMARY KEY (one or more columns)  
);
```

- Defines the structure of a table, including columns, data types, and constraints like primary keys.

DDL: CREATE TABLE Example

Purpose: Demonstrates creating the Football table.

Example

```
-- Create the Football table
CREATE TABLE Football (
  PlayerID INT PRIMARY KEY IDENTITY(1,1),
  Name VARCHAR(100) NOT NULL,
  Position VARCHAR(50) NOT NULL,
  Age INT NOT NULL,
  Nationality VARCHAR(50) NOT NULL,
  Price DECIMAL(10,2) NOT NULL,
  ContractExpiryDate DATE NULL,
  CONSTRAINT CHK_Age CHECK (Age >= 16 AND Age <= 45),
  CONSTRAINT CHK_Price CHECK (Price >= 0)
);
```

- Defines the Football table with all columns and constraints.

DDL: ALTER TABLE Statement

Purpose: Modifies a table's structure (add, drop, or modify columns).

Syntax

```
ALTER TABLE table_name {ADD|DROP|MODIFY} column_name datatype;
```

Example

```
-- Add a new column for salary  
ALTER TABLE Football ADD Salary DECIMAL(10,2);
```

- Adds a Salary column to the Football table.

DDL: ALTER TABLE (Rename) Statement

Purpose: Renames an existing table.

Syntax

```
ALTER TABLE table_name RENAME TO new_table_name;
```

Example

```
-- Rename Football to Players  
ALTER TABLE Football RENAME TO Players;
```

- Renames the Football table to Players.

DDL: CREATE INDEX Statement

Purpose: Creates an index to improve query performance.

Syntax

```
CREATE UNIQUE INDEX index_name  
ON table_name (column1, column2, ... columnN);
```

Example

```
-- Create an index on Name  
CREATE UNIQUE INDEX idx_name  
ON Football (Name);
```

- Creates a unique index on the Name column in the Football table.

DDL: DROP INDEX Statement

Purpose: Removes an index from a table.

Syntax

```
ALTER TABLE table_name  
DROP INDEX index_name;
```

Example

```
-- Remove the index on Name  
ALTER TABLE Football  
DROP INDEX idx_name;
```

- Drops the `idx_name` index from the `Football` table.

DDL: TRUNCATE TABLE Statement

Purpose: Removes all rows from a table, keeping its structure.

Syntax

```
TRUNCATE TABLE table_name;
```

Example

```
-- Remove all data from Football table  
TRUNCATE TABLE Football;
```

- Deletes all rows from the Football table but retains its structure.

DDL: DROP TABLE Statement

Purpose: Deletes an existing table and all its data.

Syntax

```
DROP TABLE table_name;
```

Example

```
-- Remove the Football table  
DROP TABLE Football;
```

- Deletes the Football table and all its data.

DDL: DROP DATABASE Statement

Purpose: Deletes an existing database and all its contents.

Syntax

```
DROP DATABASE database_name;
```

Example

```
-- Delete the FootballDB database  
DROP DATABASE FootballDB;
```

- Deletes the FootballDB database and all its contents.

Purpose: Displays the structure of a table (MySQL-specific).

Syntax

```
DESC table_name;
```

Example

```
-- Show the structure of the Football table  
DESC Football;
```

- Displays the column details of the Football table in MySQL.

DML: INSERT INTO Statement

Purpose: Inserts new rows into a table to populate it with data.

Syntax

```
INSERT INTO table_name (column1, column2, ... columnN)
VALUES (value1, value2, ... valueN);
```

Example

```
-- Insert sample player data
INSERT INTO Football (Name, Position, Age, Nationality, Price, ContractExpiryDate)
VALUES
    ('Lionel Messi', 'Forward', 38, 'Argentina', 50000000.00, '2026-06-30'),
    ('Cristiano Ronaldo', 'Forward', 40, 'Portugal', 30000000.00, '2025-12-31'),
    ('Kylian Mbapp', 'Forward', 26, 'France', 150000000.00, NULL),
    ('Kevin De Bruyne', 'Midfielder', 34, 'Belgium', 80000000.00, '2027-05-31');
```

DML: UPDATE Statement

Purpose: Updates existing rows in a table.

Syntax

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ... columnN = valueN  
WHERE CONDITION;
```

Example

```
-- Update a player's age  
UPDATE Football  
SET Age = 39  
WHERE Name = 'Lionel Messi';
```

- Updates the age of the player named 'Lionel Messi' in the Football table.

DML: DELETE Statement

Purpose: Deletes rows from a table based on a condition.

Syntax

```
DELETE FROM table_name  
WHERE CONDITION;
```

Example

```
-- Delete players older than 40  
DELETE FROM Football  
WHERE Age > 40;
```

- Removes players over 40 years old from the Football table.

DQL: SELECT Statement

Purpose: Retrieves data from one or more columns in a table.

Syntax

```
SELECT column1, column2, ... columnN  
FROM table_name;
```

Example

```
-- Retrieve player names and positions  
SELECT Name, Position  
FROM Football;
```

- Selects Name and Position columns from the Football table.

DQL: DISTINCT Clause

Purpose: Returns only unique values from a column or set of columns.

Syntax

```
SELECT DISTINCT column1, column2, ... columnN  
FROM table_name;
```

Example

```
-- Retrieve unique nationalities  
SELECT DISTINCT Nationality  
FROM Football;
```

- Returns a list of unique nationalities from the Football table, eliminating duplicates.

DQL: WHERE Clause

Purpose: Filters rows based on a specified condition.

Syntax

```
SELECT column1, column2, ... columnN  
FROM table_name  
WHERE CONDITION;
```

Example

```
-- Select players older than 30  
SELECT Name, Age  
FROM Football  
WHERE Age > 30;
```

- Retrieves names and ages of players from the Football table who are over 30 years old.

DQL: AND/OR Clause

Purpose: Combines multiple conditions in a WHERE clause using logical operators.

Syntax

```
SELECT column1, column2, ... columnN
FROM table_name
WHERE CONDITION-1 {AND|OR} CONDITION-2;
```

Example

```
-- Select young forwards
SELECT Name, Position, Age
FROM Football
WHERE Position = 'Forward' AND Age < 25;
```

- Retrieves players who are forwards and under 25 years old from the Football table

DQL: IN Clause

Purpose: Filters rows where a column's value is in a specified list.

Syntax

```
SELECT column1, column2, ... columnN  
FROM table_name  
WHERE column_name IN (val-1, val-2, ... val-N);
```

Example

```
-- Select players from specific countries  
SELECT Name, Nationality  
FROM Football  
WHERE Nationality IN ('Brazil', 'Argentina');
```

- Retrieves players from Brazil or Argentina in the Football table.

DQL: BETWEEN Clause

Purpose: Filters rows where a column's value lies within a specified range.

Syntax

```
SELECT column1, column2, ... columnN
FROM table_name
WHERE column_name BETWEEN val-1 AND val-2;
```

Example

```
-- Select players with transfer fees between 10M and 50M
SELECT Name, Price
FROM Football
WHERE Price BETWEEN 10000000 AND 50000000;
```

- Retrieves players with transfer fees between 10 million and 50 million from the Football table

DQL: LIKE Clause

Purpose: Filters rows based on a pattern match in a column.

Syntax

```
SELECT column1, column2, ... columnN
FROM table_name
WHERE column_name LIKE { PATTERN };
```

Example

```
-- Select players with names starting with 'M'
SELECT Name
FROM Football
WHERE Name LIKE 'M%';
```

- Retrieves players whose names start with 'M' from the Football table.

DQL: ORDER BY Clause

Purpose: Sorts the result set by one or more columns in ascending or descending order.

Syntax

```
SELECT column1, column2, ... columnN
FROM table_name
WHERE CONDITION
ORDER BY column_name {ASC|DESC};
```

Example

```
-- Select players sorted by age descending
SELECT Name, Age
FROM Football
WHERE Age > 20
ORDER BY Age DESC;
```

DQL: GROUP BY Clause

Purpose: Groups rows with the same values in specified columns for aggregate calculations.

Syntax

```
SELECT SUM(column_name)
FROM table_name
WHERE CONDITION
GROUP BY column_name;
```

Example

```
-- Calculate total transfer fees by nationality
SELECT Nationality, SUM(Price)
FROM Football
WHERE Price > 0
GROUP BY Nationality;
```

DQL: COUNT Clause

Purpose: Counts the number of rows or non-null values in a column.

Syntax

```
SELECT COUNT(column_name)
FROM table_name
WHERE CONDITION;
```

Example

```
-- Count players by position
SELECT Position, COUNT(*)
FROM Football
WHERE Age >= 18
GROUP BY Position;
```

- Counts the number of players for each position for those 18 or older in the

DQL: HAVING Clause

Purpose: Filters grouped results based on an aggregate condition.

Syntax

```
SELECT SUM(column_name)
FROM table_name
WHERE CONDITION
GROUP BY column_name
HAVING (arithmetic function condition);
```

Example

```
-- Nationalities with total transfer fees over 100M
SELECT Nationality, SUM(Price)
FROM Football
WHERE Price > 0
GROUP BY Nationality
HAVING SUM(Price) > 100000000;
```

TCL: COMMIT Statement

Purpose: Saves all changes made in the current transaction.

Syntax

```
COMMIT;
```

Example

```
-- Save changes after inserting a player  
INSERT INTO Football (Name, Position) VALUES ('Cristiano Ronaldo', 'Forward');  
COMMIT;
```

- Commits the insertion of a new player to the Football table, making it permanent.

TCL: ROLLBACK Statement

Purpose: Undoes all changes made in the current transaction.

Syntax

```
ROLLBACK;
```

Example

```
-- Undo an insert if there's an error  
INSERT INTO Football (Name, Position) VALUES ('Neymar', 'Forward');  
ROLLBACK;
```

- Reverts the insertion of a new player in the Football table.